

Flag : `picoCTF{tw0_1$_pr!m3lc9046c4}`

📄 Résumé de la faille

Un service génère des paires de clés RSA (1024 bits) et chiffre le flag avec. Le service fournit `N`, `e=65537` et le texte chiffré. La faille : le `N` généré est **systématiquement pair** — impossible en RSA normal, où `N = p × q` doit toujours être impair (produit de deux nombres premiers, forcément impairs sauf le nombre 2).

Un `N` pair signifie que l'un des deux facteurs premiers est **2** (le seul nombre premier pair) → `N` devient trivial à factoriser, et donc tout le RSA s'effondre.

🔧 Méthodo — les étapes

- 1. Lire l'énoncé et les hints - "How much do we trust randomness?" → suspecter la génération aléatoire - "Notice anything interesting about N?" → observer N attentivement - "Try comparing N across multiple requests" → se connecter plusieurs fois et comparer
- 2. Se connecter plusieurs fois au service (`nc`) et collecter les N

```
nc verbal-sleep.picoctf.net PORT
```

→ récupérer `N`, `e`, `cyphertext` à chaque connexion (9 essais faits ici).

- 3. Analyser le code source fourni (`encryption.py`) - Confirme un RSA standard (`e=65537`, chiffrement `pow(m, e, N)` classique) - La fonction `get_primes()` (qui génère `p` et `q`) est importée d'un fichier `setup.py` non fourni → la faille est cachée là, à déduire par observation plutôt que par lecture directe du code.
- 4. Repérer l'anomalie : tester la parité de N - Un nombre premier > 2 est toujours impair → `N = p × q` doit toujours être impair. - En regardant le dernier chiffre de chaque `N` récupéré : **tous pairs**, systématiquement (pas un cas rare — un vrai bug de génération). - Conclusion : `p = 2` à chaque génération.
- 5. Résoudre avec `dcode.fr` (Chiffre RSA) - Aller sur `dcode.fr/chiffre-rsa` - Renseigner les 3 valeurs connues : **C** (cyphertext), **E** (65537), **N** - Laisser les autres champs vides (D, P, Q, Phi — inconnus) - Cliquer "**Calculer/Déchiffrer**" - L'outil teste automatiquement plusieurs attaques et affiche le résultat : décomposition en facteurs premiers réussie → D calculé → déchiffrement → flag en clair.

🤖 Pourquoi dcode.fr a réussi à casser ce RSA

`dcode.fr/chiffre-rsa` n'est pas magique : il **teste plusieurs attaques RSA connues automatiquement** dès qu'on lui donne N, e et C sans clé privée. Dans le panneau de résultats, on voit sa démarche : - "Attaque de Wiener : échec" → tentée, ne fonctionne pas ici - "P,Q calculé(s) avec N (Décomposition en Facteurs premiers (Auto-Limitée))" → **réussi** - "D calculé(s) avec P,Q,E" → réussi - "Déchiffrement avec C,D,N" → réussi

La raison pour laquelle la factorisation a marché instantanément : factoriser un `N` normal (produit de deux grands nombres premiers aléatoires) est un problème mathématiquement très difficile — c'est précisément ce qui rend RSA sûr. Mais ici, un des deux facteurs est **2**, le plus petit nombre premier possible. Un simple test de divisibilité par les petits nombres premiers (2, 3, 5, 7...) — que `dcode.fr` effectue en "auto-limité" avant d'essayer des méthodes plus lourdes — suffit à trouver immédiatement que `N` est divisible par 2. Une fois `p=2` trouvé, `q = N/2` est immédiat, et tout le reste (calcul de `d`, déchiffrement) suit mécaniquement.

Donc dcode.fr n'a pas eu besoin d'une attaque sophistiquée : la faiblesse (facteur premier = 2) est tellement grossière qu'un test de factorisation basique et rapide suffit à la détecter, ce que l'outil fait automatiquement avant même de tenter des méthodes plus avancées.

📦 Outils utilisés

Outil	Rôle
<code>nc</code> (netcat)	Récupérer plusieurs jeux de N/e/cyphertext
Lecture du code source	Confirmer le protocole RSA et repérer où chercher la faille
<code>dcode.fr/chiffre-rsa</code>	Factorisation automatique + calcul de D + déchiffrement, sans script à écrire

✅ Réflexes à retenir

- Un `N` RSA est **toujours impair** en théorie — un `N` pair est une anomalie immédiate à exploiter (facteur = 2).
- Vérifier la **parité** d'un nombre est le test le plus simple possible (regarder le dernier chiffre) — à tester avant des analyses plus complexes (PGCD entre plusieurs N, attaque de Wiener...).
- `dcode.fr/chiffre-rsa` est un bon réflexe de premier essai sur un challenge RSA avec seulement N/e/C connus : il teste plusieurs attaques classiques automatiquement (Wiener, factorisation directe...) sans avoir à écrire de script.
- Un code source fourni peut ne pas tout révéler directement (ici, `setup.py` avec la vraie faille était absent) — parfois il faut compléter par de l'observation empirique du comportement réel.

🔗 Dans la vraie vie

Ce challenge illustre un principe fondamental : **la sécurité de RSA repose entièrement sur la difficulté de factoriser N**. Cette difficulté n'existe que si `p` et `q` sont deux grands nombres premiers choisis aléatoirement et indépendamment. Toute faiblesse dans leur génération (mauvais générateur aléatoire, biais, facteur prévisible comme ici) réduit à néant toute la sécurité — peu importe que N fasse 1024 bits ou plus.